| | |
|---|---|
| --yjit-trace-exits | Produces a stack trace when YJIT exits compile mode. Also enables --yjit-stats. |

## TruffleRuby

Now we move away from runtime engines that ship with the Ruby virtual machine and toward other Ruby implementations that run in various other environments. In general, these implementations offer performance improvements at the cost of lagging behind new features in Ruby and also limiting access to the Ruby ecosystem, since not all gems can run in the other environments.

GraalVM[3] is a virtual machine environment implemented in Java and designed to be a high-performance cross-language virtual environment. TruffleRuby[4] is a Ruby implementation built on top of GraalVM. TruffleRuby provides very high performance relative to the standard Ruby implementations; however, it is not completely compatible with standard Ruby.

If you use a Ruby version manager to install Ruby, it likely incudes TruffleRuby as one of it's downloadable options—this is true of rbenv, which is the recommended setup in this book, as well as the other commonly used version managers such as RVM, asdf, and chruby. As we write this, the current version is 22.3.0, but new versions come out on a regular schedule, so the current version number is likely higher for you when you read this in the future.

In rbenv, you can install TruffleRuby just as you would any other Ruby implementation (`rbenv install truffleruby-22.3.0` or `rbenv install truffle ruby+graalvm-22.3.0`). In the default version, TruffleRuby has already been compiled to a native executable. The `+graalvm` version runs inside the Graal Java virtual machine at runtime, just like any other Java application. The GraalVM version gives you better compatibility with other language tools at the cost of some short-term performance degredation. The TruffleRuby team recommends the native configuration for shorter-running programs, smaller memory use, or any situation where startup time is important. The GraalVM version is recommended for better performance in a long-lived process.

Once you have TruffleRuby installed, it runs just like regular Ruby. All the CLI commands for normal Ruby apply, except for ones that cover, say, JIT compilers that don't exist in TruffleRuby.

A great feature of TruffleRuby is that it tries to re-implement the core library in Ruby (most of the core library in the main runtime is written in C). The idea is that TruffleRuby is better off having the core in Ruby where the same optimizations can be used, then having them outsourced to a nominally faster language. You can find those definitions at https://github.com/oracle/truffleruby/tree/master/src/main/ruby/truffleruby/core, and they can be helpful in understanding how some of Ruby's core methods work.

One downside of using a third-party Ruby runtime is that it is not guaranteed to be compatible with the current versions of CRuby. TruffleRuby 22.3.0 is based on Ruby 3.1, and according to one comparison of Ruby runtimes,[5] it passes 97% of the ruby-spec test suite, with most of the failures being related to the CLI or edge cases in the language. If you are

3. https://www.graalvm.org
4. https://www.graalvm.org/ruby
5. https://eregon.me/rubyspec-stats